A Synthetic Multi-Source Database Audit Dataset for Behavioral Insider Threat Detection

Nguyen Van Sang, Pham Thi Xuan, Hoang Tuan Hao
Le Quy Don Technical University

Abstract

Insider threats in relational database systems are difficult to detect because malicious users often operate with legitimate credentials and perform actions that may appear similar to normal database activities. However, existing public insider threat datasets mainly focus on endpoint, authentication, file, web, or email logs, while database-native evidence such as SQL command types, accessed tables, permission outcomes, affected rows, and table sensitivity levels are rarely represented. To address this limitation, this study proposes a database-backed synthetic dataset generation approach for insider threat analysis in relational database environments. The proposed dataset is built on an extended TPC-H schema, where standard business tables are combined with enterprise-oriented tables such as employee, payroll, account, monitoring, and security-related data. Users are assigned organizational roles and clearance levels, while database tables are assigned sensitivity levels. Normal behavior is generated through executable SQL sessions under role-based access constraints. In addition, four database-oriented insider threat scenarios are designed, including unauthorized exploration, sensitive data theft, slow-rate data leakage, and data staging and aggregation. The generated audit logs are exported as CSV files and characterized using descriptive statistics, including SQL command distribution, table access frequency, permission outcomes, temporal activity patterns, and attack deviation from the normal baseline. The results provide a structured and explainable dataset foundation for future database insider threat detection and graph-based learning experiments.

Keywords: insider threat detection, database audit log, synthetic dataset, relational database, TPC-H, SQL behavior

1. Introduction

In modern data-driven organizations, relational database management systems (RDBMSs) play a central role in storing and processing critical business information, including financial transactions, operational records, intellectual property, and personally identifiable information (PII). As organizational activities increasingly depend on database-driven applications, database audit logs have become an important source of evidence for monitoring security-relevant behavior at the data access layer. Unlike network-level or endpoint-level telemetry, database audit logs capture fine-grained interactions between users, sessions, SQL statements, and relational objects, making them particularly valuable for detecting abnormal access patterns and potential insider threats.

Insider threats represent a persistent challenge in enterprise security because malicious or negligent actors often operate with legitimate credentials and authorized system access. Such users may include current employees, former employees with unrevoked privileges, contractors, partners, or temporary personnel. Compared with external attackers, insiders may possess contextual knowledge of organizational workflows, access control policies, database schemas, and auditing mechanisms. As a result, malicious database activities may appear superficially similar to routine business operations and can be difficult to distinguish from legitimate transactions, especially when attacks are conducted gradually or during periods of normal workload variation.

Although graph neural networks (GNNs) and temporal heterogeneous graph learning have shown promise for modeling complex behavioral dependencies, their application to database insider threat detection remains limited by the lack of public, database-centric audit log datasets. Existing insider threat benchmarks, such as the CERT Insider Threat Dataset, primarily focus

on host-level or endpoint-level activities, including logon events, file operations, removable device usage, HTTP access, and email metadata. These datasets are valuable for studying general insider behavior, but they do not directly represent database-specific entities and interactions such as SQL sessions, query templates, accessed tables, permission-denied events, affected row counts, or relation-level sensitivity.

This limitation is especially important because database attacks often involve structured query behavior rather than isolated file or network events. For example, an insider may explore metadata tables, repeatedly query sensitive relations, aggregate records across multiple tables, or stage extracted data through temporary objects before exfiltration. Such behaviors are difficult to represent using flat event sequences alone because they involve multiple interacting entities and relation-specific semantics.

A second obstacle is the privacy and confidentiality constraint associated with real database audit logs. Production audit logs may reveal proprietary schemas, business logic, customer data, employee information, and internal security policies. Therefore, sharing real enterprise database logs for research purposes is often infeasible due to legal, ethical, and commercial restrictions. This creates a practical data scarcity problem: researchers require realistic and semantically rich datasets for evaluating graph-based detection methods, while organizations are generally unable to release production audit traces.

To address this gap, this study proposes a database-aware synthetic data generation framework for insider threat research, in which normal and malicious activities are executed on an actual relational database environment constructed from an extended TPC-H schema. Unlike purely random log generation, the framework produces audit records grounded in executable SQL operations against real database tables, preserving database-specific evidence such as SQL command types, accessed relations, execution outcomes, permission-denied events, and affected row counts. Normal behavior is simulated according to role-specific access rules, clearance levels, table sensitivity levels, working-hour patterns, and session-level activity profiles, while multi-phase insider attack scenarios are injected on top of this baseline to reflect database-oriented kill-chain behaviors, including reconnaissance, preparation, execution, and cover phases.

The main contributions of this study are as follows.

First, we design a multi-source simulation pipeline that generates synchronized database audit logs, system monitoring records, and contextual security alert logs.

Second, we construct four database-oriented insider threat scenarios that capture different attack behaviors, including unauthorized exploration, sensitive data theft, slow-rate data leakage, and data staging and aggregation.

Finally, we evaluate the generated dataset through statistical and behavioral validation, including descriptive log statistics, normal-versus-attack comparisons, and behavioral deviation analysis.

2. Related work

2.1 Insider Threat and Security Datasets

Public benchmark datasets have played an important role in the development and evaluation of insider threat detection methods. Among them, the CERT Insider Threat Dataset is one of the most widely used synthetic corpora for studying malicious insider behavior. CERT variants, such as r4.2 and r6.2, simulate enterprise activity over a multi-month timeline and provide event records related to logon and logoff activities, removable device usage, file access, web browsing, and email communication. These datasets also include ground-truth labels for predefined insider threat scenarios, making them useful for supervised and semi-supervised anomaly detection research.

However, when considered from the perspective of relational database security, these datasets have several limitations. CERT-style logs mainly describe endpoint and application-level activities rather than database-native transactions. For example, data exfiltration is often represented through file copying, removable device usage, or web uploads, whereas many real enterprise data breaches originate from structured queries executed directly against database systems. As a result, important database-level forensic signals, such as SQL command type, accessed relation, query template, permission outcome, affected row count, and table sensitivity level, are not explicitly modeled.

Other public cybersecurity datasets, such as the Los Alamos National Laboratory (LANL) authentication dataset, provide large-scale records of authentication and network activity. These datasets are valuable for studying lateral movement, authentication anomalies, and network-level security behavior. Nevertheless, they do not capture the semantic structure of database interactions. In particular, they do not model the relationships among database users, SQL sessions, query templates, and relational objects. This limits their applicability for evaluating database audit models that require fine-grained access semantics and relation-level context.

2.2. Database Audit Log Datasets

Database activity monitoring focuses on recording and analyzing operations performed inside database management systems. Audit logs can provide detailed evidence of SQL execution, privilege usage, data manipulation, schema changes, and access-control violations. In practice, such logs may be collected through native database auditing mechanisms, audit extensions, or database activity monitoring tools.

Despite their importance, public database audit log datasets are still scarce. Real audit logs often contain sensitive information about organizational schemas, business processes, users, customers, and security configurations. Therefore, many studies rely on private datasets, laboratory testbeds, or small simulated environments. These datasets are difficult to reuse and do not support fair comparison across different methods.

Existing synthetic SQL log datasets also have limitations. Many of them are small in scale, lack role-based access behavior, or do not model long-term attack progression. They often ignore important database security factors such as user clearance, table sensitivity, permission outcomes, affected rows, and working-hour patterns. As a result, they are not sufficient for evaluating database-oriented insider threat scenarios.

2.3 Synthetic Log Generation and Statistical Validation

Synthetic log generation has become an important approach for addressing data scarcity in cybersecurity research. Early approaches commonly relied on rule-based simulation, stochastic processes, or Markov models to generate chronological event sequences. These methods can reproduce basic event frequencies and temporal ordering, but they may not fully capture the behavioral constraints of enterprise database environments, such as role-based access rules, table sensitivity levels, permission outcomes, and working-hour patterns.

For synthetic security datasets to be useful, the generated logs should not only contain labeled attack events but also preserve realistic behavioral differences between normal and malicious activities. Therefore, statistical validation is an important step in assessing dataset quality. Descriptive statistics, normal-versus-attack comparisons, permission-denied rates, sensitive table access frequencies, after-hours activity distributions, and session-level behavioral features can be used to examine whether the generated dataset reflects plausible operational patterns and meaningful anomaly signals.

This study follows this direction by focusing on database-backed synthetic audit log generation and statistical behavioral validation. The proposed framework generates normal database activities through executable SQL operations on an extended TPC-H relational schema and injects multi-phase insider attack scenarios on top of this baseline. The resulting dataset is evaluated using descriptive log statistics, behavioral deviation analysis, and low-dimensional visualization of session-level feature representations.

3. Proposed Dataset Generation Method

3.1. TPC-H-Based Enterprise Schema

The proposed dataset is generated from an executable relational database rather than from independently created log records. The generation process first initializes the database schema, populates business and enterprise tables, and then executes simulated user activities against the database. As a result, audit records are linked to actual database objects and execution outcomes.

The core schema is based on TPC-H, including business tables such as **customer**, **orders**, **lineitem**, **supplier**, **part**, **partsupp**, **nation**, and **region**. These tables provide the normal business data layer for simulating routine enterprise queries. To support database insider threat scenarios, the schema is extended with additional enterprise-oriented tables, including **employee, payroll, account, system monitoring, and security-related** tables.

Each table is assigned a sensitivity level. TPC-H business tables are treated as lower-sensitivity relations, while enterprise internal tables such as payroll, account, monitoring, and security tables are assigned higher sensitivity levels. This sensitivity assignment is used by the access-control logic during workload generation. Therefore, the schema provides both ordinary business objects and restricted objects needed for simulating normal and suspicious database access.

3.2. Role, Clearance, and Sensitivity Modeling

The simulation uses a role-clearance-sensitivity model to control user behavior. Each user is assigned an organizational role and a clearance level. Each table is assigned a sensitivity level. For every SQL operation, the system checks whether the user is allowed to perform the requested command on the target table.

The access decision is based on four elements: user role, user clearance level, table sensitivity level, and SQL command type. This prevents the workload from being generated uniformly at random. Instead, each user can only perform operations that are consistent with the predefined organizational policy, except when the simulation intentionally records denied or abnormal access attempts.

Different roles are associated with different expected database behaviors. Sales users mainly access customer and order-related tables. Human resources users access employee-related records. Accountants access financial or payroll-related data within their permitted scope. Data analysts mainly perform read-oriented queries. Database administrators have broader access privileges for maintenance and monitoring operations.

The output of this step is a permission-aware activity model. Each SQL event can be marked as successful or denied according to the access policy. This is important because the generated audit log preserves not only what users attempted to access, but also whether the operation was allowed by the simulated enterprise policy.

3.3. Normal SQL Workload and Session Behavior Simulation

Normal behavior is generated through user sessions. A session represents one database connection period in which a user executes one or more SQL operations. This design allows the dataset to capture both event-level behavior and session-level behavior.

For each simulated session, the system selects a user, determines the user's role, chooses role-appropriate target tables, generates SQL operations, checks permission rules, executes or records the operation outcome, and writes the corresponding audit record. The generated workload includes common SQL operations such as SELECT, INSERT, and UPDATE, depending on the user role and table permission. Read-oriented roles mainly generate SELECT statements, while privileged roles may produce a wider range of operations.

Temporal behavior is controlled by predefined working-time frames. Each session is assigned to a time frame representing a typical enterprise activity period, such as regular working hours, lower-activity transition periods, evening access, or off-hour access. The timestamp of each session is then sampled within the selected frame. This mechanism prevents activities from being uniformly distributed across the whole day and makes the generated workload closer to realistic working patterns.

For each SQL event, the system records timestamp, user identifier, role, session identifier, SQL command type, target table, execution status, permission outcome, and affected row count. Fields such as execution result and affected rows are derived from database execution records or post-processing of audit records, rather than being assigned as unrelated random values. Therefore, the generated CSV logs remain connected to actual simulated database behavior.

3.4. Database-Oriented Attack Scenario Design

Attack behavior is generated by injecting abnormal SQL activities into the normal workload. The attacker is modeled as an authenticated database user, meaning that malicious behavior occurs through database sessions and SQL operations rather than through external network intrusion. This design is suitable for insider threat simulation because insiders may already have partial legitimate access to the database.

The attack generation follows four database-oriented scenarios. Each scenario is mapped to a different behavioral objective and produces different audit-log evidence.

S1 Unauthorized Data Exploration simulates early reconnaissance behavior. The attacker explores database objects beyond the expected scope of their role. This may include querying

rarely used tables, accessing a wider range of relations than normal users in the same role, or attempting to read restricted tables. The main evidence generated by this scenario includes unusual table access, sensitive-table access attempts, and permission-denied records.

S2 Sensitive Data Theft simulates direct extraction of confidential information. The attacker performs queries on high-sensitivity tables such as employee, payroll, account, monitoring, or security-related tables. Compared with normal behavior, this scenario may generate higher affected row counts, repeated access to sensitive tables, or multiple sensitive queries within the same session.

S3 Slow-Rate Data Leakage simulates stealthy and low-volume extraction. Instead of retrieving many records at once, the attacker accesses small portions of sensitive data across multiple sessions or days. Individual events may appear close to normal, but the repeated temporal pattern creates an abnormal long-term signal.

S4 Data Staging and Aggregation simulates the preparation of data for later misuse. The attacker collects, combines, or aggregates useful information from multiple related tables. This scenario focuses on staging and consolidation of sensitive information, not destructive sabotage. Its evidence may include repeated access to related sensitive tables, cross-table aggregation behavior, and abnormal session patterns involving multiple database objects.

All attack scenarios are recorded in the same audit-log structure as normal behavior. Therefore, attack and normal events can be compared using database-level features such as accessed table, SQL command type, permission outcome, affected row count, session behavior, and timestamp.

4. Experimental Setup

4.1. Simulation Configuration

The experiment is implemented as a database-backed simulation pipeline. It first creates an executable PostgreSQL database based on the TPC-H schema and additional enterprise-specific tables, with TPC-H data loaded using **dbgen**. Python scripts then generate users, roles, sessions, normal SQL workloads, and attack SQL workloads according to predefined role, clearance, table sensitivity, and permission rules. The generated SQL statements are executed on PostgreSQL, and the resulting activities are captured through PostgreSQL audit mechanisms. Attack scenarios are injected into selected sessions and days using configured attacker accounts.

Python communicates with PostgreSQL through the psycopg2 library, which is used to submit generated SQL statements and control session-level execution. PostgreSQL logging and pgAudit configurations then capture the executed statements and audit-relevant metadata for later parsing and enrichment.

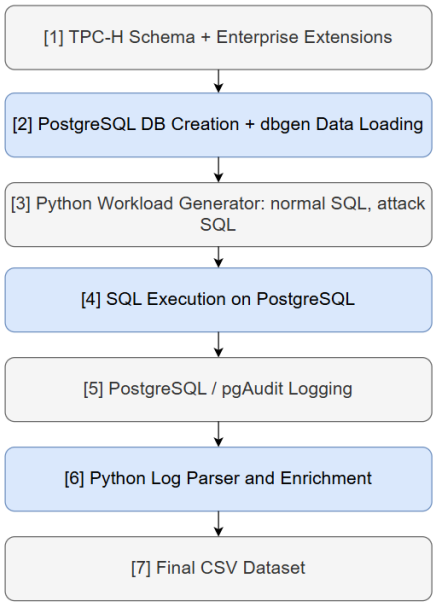

Figure 1. Experimental simulation pipeline.

Normal and attack activities are generated through database sessions. For each session, the generator selects a user, assigns a timestamp based on predefined working-time frames, selects role-appropriate target tables, generates SQL operations, checks access permission, and records the execution result. The use of working-time frames allows normal activities to concentrate in regular business periods, while lower-frequency or abnormal activities may appear in evening or off-hour frames.

The attack generation is executed on top of the normal baseline. This means that attacker users may still have normal activities, but additional abnormal SQL behaviors are injected according to the designed scenarios: unauthorized exploration, sensitive data theft, slow-rate data leakage, and data staging and aggregation. This setting is suitable for insider threat simulation because malicious insiders often operate through legitimate accounts and mix abnormal behavior with routine database usage.

4.2. Generated Log Files and Exported CSV Data

After simulation, the generated records are exported into CSV files for inspection and statistical analysis. The main output is the database audit log. Each audit record stores SQL-level evidence, including timestamp, user identifier, role, session identifier, SQL command type, accessed table, permission outcome, execution status, affected row count, and anomaly label where applicable.

The system also exports supporting files to describe the simulation context. User-related files store user roles and clearance levels. Table-related metadata stores table categories and sensitivity levels. Security-related logs record abnormal or attack-related events, while monitoring-related records provide session or system-context indicators generated from the simulation and audit information. An attacker list is also exported to identify the configured malicious users used in the attack generation stage.

No.	CSV File	Description	Main attributes
1	table_metadata.csv	Metadata of database tables and their sensitivity levels	table_name, schema, security_level, level_label, row_count
2	table_relations.csv	Structural relationships among database tables	source_table, source_column, target_table, target_column, relation_type, cardinality
3	user_metadata.csv	Simulated database users, roles, clearance levels, and permission matrix	db_user, job_role, clearance_level, db_role, is_privileged, perm_L1, perm_L2, perm_L3, perm_L4, perm_L5
4	attack_users.csv	Configured attacker accounts and attack time ranges	scenario, db_user, job_role, start_day, end_day, start_date, end_date, phase_plan
5	audit_YYYY-MM-DD.csv	Daily SQL-level database audit log	event_id, timestamp, db_user, session_id, sql_statement, tables_involved, status, rows_affected, graph_frame, sim_date, is_after_hours, session_start_time, session_ip, session_event_index, scenario_type

6	system_monitor_log.csv	Monitoring log generated for database sessions and runtime context	monitor_id, event_time, db_user, session_id, cpu_utilization, memory_utilization, active_connections, disk_read_kb, disk_write_kb, backup_status
7	system_security_log.csv	Security-relevant events associated with sensitive or abnormal activity	alert_id, event_time, db_user, event_type, severity_index, source_ip, target_object, related_event_id, description, related_session_id

Table 1. Exported CSV files and main attributes

Table 1 summarizes the CSV files exported by the simulation pipeline. The daily audit files are the primary source for SQL-level behavioral analysis, while the metadata files describe users, roles, permissions, table sensitivity, and table relationships. The monitoring and security logs provide additional runtime and security context.

4.3. Statistical Features and Evaluation Criteria

The experiment evaluates the generated dataset through CSV-based statistical analysis. The purpose is to verify whether the generated logs are structurally correct, consistent with the access-control design, and behaviorally meaningful for insider threat research.

The analysis uses four groups of features. Dataset-scale features include the number of users, roles, tables, sessions, SQL events, and simulated days. Access-control features include role distribution, clearance distribution, table sensitivity distribution, permission success rate, and denial rate. SQL behavior features include command-type distribution, accessed-table frequency, affected row count, and number of events per session. Temporal and attack-related features include event distribution by day, event distribution by time frame, sensitive-table access frequency, and normal-versus-attack behavior differences.

The evaluation follows three criteria. First, the generated records should follow the predefined role-clearance-sensitivity rules. Second, normal behavior should show plausible enterprise database patterns, such as role-specific table access and concentration of activity in working-time frames. Third, attack records should create observable deviations from the normal baseline, especially in sensitive-table access, permission-denied events, affected row counts, repeated access patterns, and activity across attack phases.

5. Experimental Results

5.1. Dataset Overview and Scale

The simulation generated 90 days of database activity, with 60 days containing only normal SQL operations and the remaining 30 days including attack scenarios, covering the period from January 1 to March 31, 2026. The final dataset contains 79 users, 17 job roles, 14 database tables, 13,380 sessions, and 95,817 audit events. Among these events, 94,770 were successful, 1,047 were denied, and 364 were attack-related events.

These results show that the dataset is highly imbalanced, with normal events forming the majority of the workload. This setting is consistent with insider threat analysis, where malicious activities usually represent only a small fraction of total system behavior.

No.	Metric	Value
1	Simulation period	90 days

2	Start - End Date	2026/01/01 - 2026/03/31
3	Number of database users	79
4	Number of job roles	17
5	Number of database tables	14
6	Number of sessions	13,380
7	Number of audit events	95,817
8	Successful events	94,770
9	Denied events	1,047
10	Attack-related events	364
11	System monitoring records	14,224
12	System security records	8,813

Table 2. Overall dataset scale.

5.2. User, Role, Clearance, and Table Sensitivity Distribution

The user metadata contains 79 users assigned to 17 job roles. Most users belong to ordinary business roles such as sales staff, accountant, procurement staff, developer, human resources staff, and data analyst. Only 6 users are marked as privileged, while 73 users are non-privileged.

The clearance distribution is also uneven. There are 28 users at clearance level 1, 34 users at level 2, 11 users at level 3, and 6 users at level 4. This reflects an enterprise-like setting in which most users have low or medium access privileges.

The database contains 14 tables across five sensitivity groups: public, business, internal sensitive, confidential, and restricted. Sensitive tables include payroll, database account, system monitoring, and system security tables.

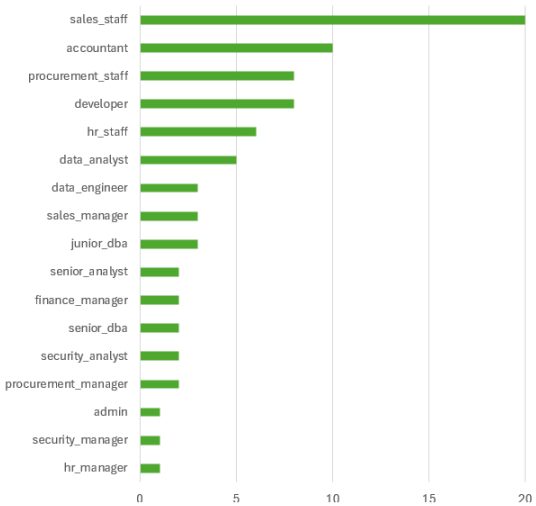

Figure 2. User distribution by role.

5.3. Audit Log Statistics and Affected Row Analysis

The audit log contains 95,817 SQL-level events. Successful operations account for most records, while denied events represent access attempts that violate role, clearance, or sensitivity rules. The existence of 1,047 denied events confirms that permission outcomes are explicitly captured in the dataset.

Affected row statistics also show differences across behavior types. Normal events have an average affected-row value of 15.25. In contrast, S2 execution has an average of 158.00 affected rows, while S4 execution reaches an average of 1,072.10 and a maximum of 5,000 rows. This indicates that affected rows can help distinguish routine queries from high-volume extraction or staging behavior.

No.	Scenario	Events	% total events	Main behavioral signal
1	Normal	94,509	98.64	Routine role-based database access
2	Noise denied	944	0.99	Background denied access attempt
3	S1 Unauthorized exploration	99	0.10	Denied probing and unusual table a
4	S2 Sensitive data theft	157	0.16	Sensitive-table access and larger row retrieval
5	S3 Slow-rate data leakage	68	0.07	Low-volume repeated access over time
6	S4 Data staging and aggregation	79	0.08	High-volume staging and aggregation behavior

Table 3. Scenario-level audit statistics

5.4. Temporal and Role-Based Access Pattern Analysis

The dataset covers a continuous 90-day timeline with 13,380 sessions. Daily event counts can be used to inspect workload stability and identify attack-injection periods. The generated records also include time-frame information, allowing activities to be analyzed by predefined working-time frames.

For example, in the sample audit file of January 1, 2026, most events occur in regular activity frames such as G2, G3, and G4. Only 24 events are marked as after-hours, while 1,290 events occur during non-after-hours periods. This indicates that the generated workload follows the intended temporal design.

Role-based analysis can be performed by joining audit logs with user metadata and table metadata. This allows the dataset to show whether each role accesses expected table groups and whether denied operations are concentrated in specific roles or sensitivity levels.

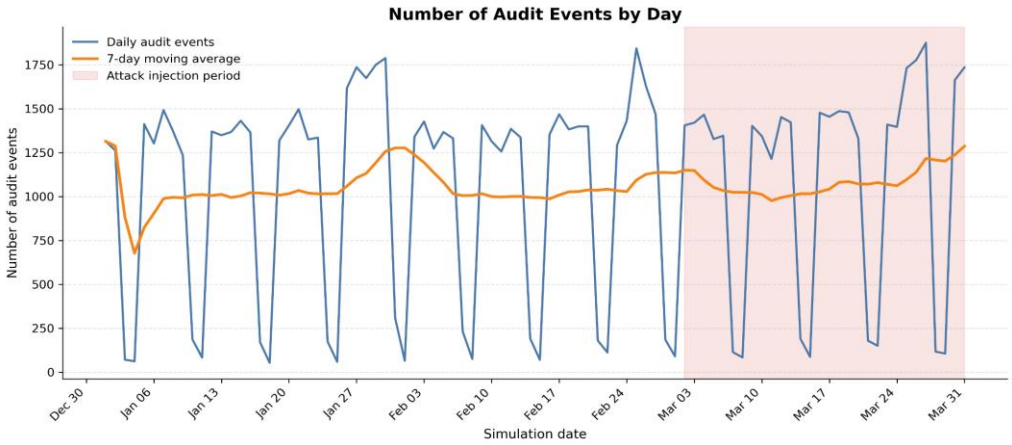

Figure 6. Number of audit events by day

5.5. Normal-versus-Attack Behavioral Deviation

The dataset includes four attack scenarios: S1 unauthorized data exploration, S2 sensitive data theft, S3 slow-rate data leakage, and S4 data staging and aggregation. Attack events are injected between March 2, 2026 and March 31, 2026.

The scenarios produce different deviation patterns. S1 contains both successful and denied exploration attempts. S2 shows higher affected-row values during execution, indicating sensitive data extraction. S3 produces lower-volume but repeated access over time. S4 produces the largest affected-row values, which is consistent with data staging and aggregation.

These results show that attack events are not only labeled differently, but also differ from normal behavior in measurable database-level features such as permission outcome, affected rows, sensitive-table access, and temporal distribution.

6. Conclusion and Discussion

6.1. Dataset Correctness and Behavioral Realism

The experimental results indicate that the generated dataset follows the intended access-control design. Each audit event is associated with a user role, clearance level, accessed table, permission outcome, and scenario type. The existence of both successful and denied events shows that access decisions are explicitly represented rather than ignored during log generation.

The dataset also reflects realistic behavioral imbalance. Normal activity accounts for 98.64% of all audit events, while attack events represent only a small fraction of the dataset. This distribution is consistent with insider threat detection, where malicious behavior is rare compared with routine database usage.

Behavioral realism is further supported by role-specific and temporal patterns. Users do not access all tables uniformly; their activities depend on organizational roles and table sensitivity. In addition, session timestamps are generated using working-time frames, allowing the dataset to reflect normal working-hour activity and lower-frequency off-hour access.

6.2. Interpretation of Attack Scenarios

The four attack scenarios produce different observable patterns in the audit logs. S1 Unauthorized Exploration is mainly characterized by denied probing attempts and unusual access to restricted tables. This reflects early-stage reconnaissance behavior, where the attacker attempts to discover accessible database objects.

S2 Sensitive Data Theft shows larger affected-row values than normal behavior. Its average affected rows reach 38.68, compared with 15.25 for normal events. This indicates that the scenario captures direct retrieval of sensitive records.

S3 Slow-Rate Data Leakage produces smaller affected-row values, with an average of 3.60 rows. This is consistent with stealthy leakage behavior, where the attacker retrieves small amounts of data repeatedly to avoid obvious volume spikes.

S4 Data Staging and Aggregation produces the strongest volume-based deviation. Its average affected rows reach 547.52, with a maximum of 5,000 rows. This pattern reflects aggregation or staging behavior, where the attacker prepares a larger amount of data for potential misuse.

These differences show that the scenarios are not only distinguished by labels, but also by measurable database-level evidence such as permission outcomes, affected rows, accessed tables, and temporal behavior.

6.3. Suitability for Future Graph-Based Learning

Although this study focuses on CSV-based dataset generation and statistical validation, the exported logs contain entities and relationships suitable for future graph construction. Users, sessions, SQL statements, and database tables can be represented as heterogeneous node types. Their relationships can be derived from audit records, such as user-session, session-SQL, and SQL-table interactions.

The dataset also contains useful attributes for graph features. User nodes can include role and clearance level. Session nodes can include query count, time frame, permission outcomes, and scenario type. SQL-related nodes or edges can include command type, execution status, and affected rows. Table nodes can include sensitivity level and table category.

The timestamp field further allows the dataset to be divided into temporal windows or snapshots. Therefore, although graph learning is not evaluated in the current paper, the dataset provides a practical foundation for future temporal heterogeneous graph-based insider threat detection.

Another advantage of the proposed pipeline is its extensibility. Since the dataset is generated from configurable users, roles, permissions, time frames, SQL workloads, and attack scenarios, the pipeline can be adjusted to produce different log distributions for different research purposes. New roles, sensitive tables, attack phases, or workload rules can be added without changing the overall generation process. Therefore, the current pipeline can serve as a foundation for developing a more complete and application-oriented database insider threat dataset in future work.

6.4. Limitations

This study has several limitations. First, the evaluation is based on descriptive statistics and behavioral validation rather than machine learning performance. Therefore, the results demonstrate dataset correctness and behavioral usefulness, but they do not yet measure detection accuracy.

Second, the simulated workload is controlled by predefined role rules, clearance levels, table sensitivity levels, and time-frame settings. This makes the dataset explainable and reproducible, but it may not cover all variations of real enterprise database behavior.

Third, the dataset focuses on database-level evidence. It does not include network traffic, operating system events, or endpoint activity. Therefore, attacks that bypass the database query layer are outside the scope of this study.

Finally, graph construction and graph learning evaluation remain future work. The current paper focuses on generating and validating the database audit dataset before applying more complex detection models.

References

[1] Stolfo, S. J., et al. (2008). Fighting Insider Threats. *Proc. of the interaction of insider threats and macroeconomics.*

[2] Eldardiry, H., et al. (2016). Multi-source Fusion for Insider Threat Detection. *AISec*, 45-56.

[3] Legg, P. A., et al. (2017). Automated Insider Threat Detection. *ACM Computing Surveys*, 50(4), 1-36.

[4] Ferraiolo, D. F., & Kuhn, D. R. (1992). Role-Based Access Control. *15th NIST-NCSC*, 554-563.

[5] Sandhu, R. S., et al. (1996). Role-Based Access Control Models. *IEEE Computer*, 29(2), 38-47.

[6] Zhang, X., et al. (2020). Enhanced RBAC for Healthcare Data. *Journal of Medical Systems*, 44(3), 1-12.

[7] Sandhu, R. S. (1993). Lattice-Based Access Control Models. *IEEE Computer*, 26(11), 9-19.

[8] Chen, Y., et al. (2019). Temporal Pattern Analysis for Fraud Detection. *IEEE TKDE*, 31(8), 1452-1465.

[9] Liu, J., et al. (2021). Time Series Anomaly Detection in DB Logs. *ACM SIGKDD*, 2345-2356.

[10] Strom, B. E., et al. (2018). MITRE ATT&CK: Design and Philosophy. *MITRE Technical Report*.

[11] Theis, M., & White, J. (2022). Mapping Insider Threat to MITRE ATT&CK. *Journal of Cybersecurity Research*, 7(2), 89-104.

[12] Transaction Processing Performance Council. (2023). TPC Benchmark H (Decision Support) Standard Specification Revision 3.0.0. TPC Specification Report.

[13] Poosankam, P., et al. (2009). Debugging Access Control Policies in Relational Databases. IEEE Transactions on Software Engineering, 35(6), 790-805.

[14] Kamra, A., et al. (2008). Detecting Anomalous Access Patterns in Relational Databases. VLDB Journal, 17(5), 1063-1077.

[15] Sallam, A., et al. (2019). An Anomaly Detection Framework for Database Audit Logs Using Structural Profiling. Information Systems, 84, 112-128.

[16] Shannon, C. E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal, 27(3), 379-423.

[17] Kullback, S., & Leibler, R. A. (1951). On Information and Sufficiency. The Annals of Mathematical Statistics, 22(1), 79-86.

[18] Van der Maaten, L., & Hinton, G. (2008). Visualizing Data using t-SNE. Journal of Machine Learning Research, 9(11), 2579-2605.